dora-studio: Web GUI for Dataflow Management in Dora

Proposal Summary

This project will build dora-studio, a lightweight web-based GUI for viewing, editing, running, and monitoring Dora dataflows. Dora currently relies mainly on CLI workflows, which are efficient for experienced users but less intuitive for onboarding, debugging, and demos. The project will provide a basic Rust control API for listing, starting, stopping, and inspecting dataflows, together with a Vue-based web UI that includes a dashboard, a simple visual editor, and a live status view. The goal is to deliver a practical proof of concept that closes the workflow loop of inspecting a dataflow, making basic edits, running it, and observing runtime state. Expected deliverables include the control API, the web UI for running dataflows, a simple visual editor, live status monitoring, and documentation.

Project size

350 hours

Project Technologies

Rust, Vue.js, TypeScript, REST API, WebSocket, Frontend, Web Development, Dataflow

Project Topics

Robotics, Web, Developer Tools, Visualization, Systems, UI/UX, Monitoring, Dataflow

Github:

DGHX12345

Project Title

dora-studio: Web GUI for Dataflow Management in Dora

Applicant Bio

I am an undergraduate student in Electronic Information Science and Technology at Beijing Information Science and Technology University, currently ranked top 5 in my class.

My academic background includes sensors, signal processing, and system design, and I have practical experience building data acquisition and processing systems. In particular, I have worked on projects involving sensor integration and real-time data pipelines.

I also have strong systems programming experience (C++, Rust, and C) and have built complete data collection and inference workflows. I am currently involved in an open-source robotics project related to Dora.

In addition to technical work, I have served as the president of the student union, which has strengthened my ability to manage complex projects and collaborate effectively.

1. Abstract

Dora currently relies primarily on CLI workflows for creating, running, and managing dataflows. While the CLI is efficient for experienced developers, it creates a higher entry barrier for new contributors and makes it harder to inspect, edit, and monitor applications in an intuitive way. This project proposes **dora-studio**, a lightweight yet extensible web-based GUI for viewing, editing, running, and monitoring Dora dataflows. The purpose is not to replace the CLI entirely,

but to build a practical first version that closes the core workflow loop of **inspection, editing, execution, and live monitoring**.

A successful implementation would improve Dora's usability for new users, make debugging and demonstrations easier, and provide a strong foundation for future tooling around the Dora ecosystem.

2. Mentor-Facing Summary

This project proposes **dora-studio**, a lightweight yet extensible web GUI for viewing, editing, running, and monitoring Dora dataflows. The goal is not to replace Dora's CLI, but to complement it with a graphical entry point that improves usability, debugging, and demonstration workflows.

I plan to structure the project in two parts: a Rust backend control layer that provides minimal APIs for listing, loading, starting, stopping, and inspecting dataflows, and a Vue 3 + TypeScript frontend that provides a dashboard, run/monitor view, simple graph editor, and live event panel.

The MVP will prioritize a complete workflow loop: **inspect a dataflow, perform basic visual editing, start or stop execution, and observe live runtime status**. More advanced features, such as richer property editing or stronger validation, will be treated as stretch goals after the core workflow is stable.

I am currently involved in a Dora-related open-source robotics project and have worked on data collection and inference workflows, which gives me practical context for Dora usage. My focus is to deliver a working, demonstrable, and extensible first version rather than an overly ambitious UI with unstable scope.

3. Motivation

I am currently involved in an open-source robotics project that works with Dora, where I have been involved in data collection and inference-related work. Through this experience, I have had exposure to realistic Dora-based workflows and practical usage scenarios.

One issue becomes clear in practice: even when Dora's architecture is powerful, the first interaction experience can still be difficult, especially when a user is trying to understand and manipulate an existing dataflow.

A GUI would be particularly valuable in scenarios such as:

- onboarding new contributors by visualizing nodes and connections
- debugging perception, inference, and control pipelines by checking which components are running
- editing a dataflow with a better structural overview before rerunning it
- demonstrating Dora in a more accessible way during experiments, demos, and teaching

For this reason, I see dora-studio not merely as a frontend addition, but as a usability and observability improvement for Dora.

4. Problem Statement

Dora's CLI workflow is efficient for experienced users, but several practical limitations remain from a usability perspective.

4.1 Limited visual understanding

To understand a dataflow, users often need to inspect configuration files and documentation first. A graph representation would significantly reduce the time needed to build a mental

model of the system.

4.2 Fragmented observability

Runtime information is often spread across terminal output and logs. A centralized live status view would make runtime inspection much more efficient.

4.3 Disconnected edit-run cycle

Users edit configuration, switch back to the CLI, rerun, and then inspect output separately. A GUI can make this a continuous loop: edit, save, run, observe.

4.4 Higher onboarding cost

For demos and community adoption, a visual interface can communicate system behavior much more clearly than a purely CLI-first workflow.

5. Project Goals

The main goals of this project are:

Goal 1: Build a basic control API

Provide a backend API layer for:

- listing available dataflows
- loading dataflow definitions
- starting and stopping dataflows
- querying runtime status
- exposing basic events, logs, and error information

Goal 2: Implement a web UI for running and managing dataflows

The UI should allow users to:

- browse available dataflows
- start and stop dataflows
- inspect the current execution state
- manage the core workflow from a single interface

Goal 3: Create a simple visual editor

The editor should support:

- graph-based visualization of Dora dataflows
- basic editing of nodes and edges
- import and export of configuration
- rerunning a modified dataflow

Goal 4: Provide a live status view

The live view should display:

- dataflow lifecycle status
- node-level runtime summaries
- basic events and errors
- important runtime feedback useful for debugging

Goal 5: Write documentation and demo materials

This includes:

- setup instructions
- API documentation
- UI usage guide
- architecture notes
- example or demo material

6. Proposed Technical Approach

6.1 Architecture Overview

I plan to divide dora-studio into two clearly separated layers.

Backend Control Layer (Rust)

A lightweight control layer integrated with Dora that exposes:

- dataflow listing
- configuration loading and parsing
- start and stop controls
- runtime status queries
- event and log streaming

This layer will likely use:

- **REST APIs** for control and data access
- **WebSocket or SSE** for live updates

This separation keeps the design maintainable and makes the backend reusable beyond the GUI itself.

Frontend Layer (Vue 3 + TypeScript)

I propose using:

- **Vue 3**
- **TypeScript**
- **Vite**

This choice keeps the stack productive and manageable. It also fits the scope of a 350-hour project well.

For graph rendering and editing, I will evaluate a lightweight graph library such as Vue Flow, or use a minimal custom wrapper if that better fits Dora's dataflow needs.

Main views and components:

- Dataflow Dashboard
- Run and Monitor View
- Visual Editor
- Logs and Events Panel

6.2 MVP Scope

To ensure realistic delivery within 350 hours, I will prioritize a minimal but complete workflow.

Core MVP

- list dataflows
- inspect dataflow structure
- start and stop dataflows
- show basic runtime status
- show live events and logs
- perform basic graph editing for simple dataflows
- export or save updated configuration

Explicitly Out of Scope for the first version

- full replacement of advanced CLI workflows
- fully schema-driven editing for every possible node property
- collaborative editing
- permissions or authentication systems

- advanced deployment or orchestration management
- fully polished product-grade UI for all edge cases

This scope control is essential for keeping the project achievable.

6.3 Module Breakdown

A. Control API

Planned capabilities:

- GET /dataflows
- GET /dataflows/{id}
- POST /dataflows/{id}/start
- POST /dataflows/{id}/stop
- GET /dataflows/{id}/status
- GET /events/stream

The exact API design can be refined during the community bonding period with mentor feedback.

B. Dashboard

A unified entry point for:

- browsing available dataflows
- quick run and stop actions
- viewing recent state summaries

C. Visual Editor

The editor will prioritize:

- rendering nodes and edges
- editing graph structure
- limited property editing
- import and export of configuration

D. Live Status View

The monitoring panel will provide:

- running, stopped, and error states
- node-level summaries
- live event stream
- basic error visibility

7. Deliverables and Evaluation Criteria

By the end of the project, I expect to deliver:

1. A working basic control API
2. A web UI that can run and stop dataflows
3. A simple visual editor for viewing and editing dataflows
4. A live status view with runtime updates
5. Documentation covering setup, usage, architecture, and development notes

The project will be considered successful if:

- a user can list and start at least one existing Dora dataflow from the browser
- a user can stop a running dataflow from the UI
- the UI can display live runtime status updates
- a simple dataflow can be visualized and edited through the graph interface
- the modified configuration can be exported or saved

- another developer can follow the documentation to run the prototype locally

8. Preparation and Relevant Experience

Before submitting this proposal, I have already accumulated practical experience around Dora-related usage scenarios. I am currently involved in an open-source robotics project that works with Dora, where I have contributed to data collection and inference-related workflows. This gives me not only a documentation-level understanding of Dora, but also a practical understanding of how important dataflow management is for development, debugging, and demonstrations.

While I do not yet have merged PRs in the main Dora repository, I already have several forms of relevant preparation for this project:

- practical exposure to Dora-related dataflow usage scenarios
- understanding of the workflow loop of editing configuration, running a dataflow, and observing execution state
- systems implementation experience with Rust, C, and Java
- experience building complete data collection, training, and inference pipelines
- some frontend design experience, with a plan to use a mature frontend stack to accelerate delivery

During the application period and community bonding phase, I plan to continue the following preparation work:

- study the Dora repository and dataflow-related modules more systematically
- map out the key interaction paths of the current CLI workflow
- discuss API boundaries, MVP scope, and priorities with the mentor
- prototype or validate a minimal UI and control-layer workflow early to reduce implementation risk during the coding period

9. Timeline

Community Bonding

- study Dora repository, docs, and dataflow-related code paths
- confirm API boundaries and UI scope with the mentor
- analyze the current CLI workflow and define the MVP user journey
- prepare a more detailed design and task breakdown

Weeks 1–2: Requirements and Prototyping

- inspect dataflow formats and runtime entry points
- define the API contract
- create low-fidelity UI prototypes and page structure
- scaffold frontend and backend project structure

Milestone

- technical design prepared
- project skeletons available
- initial API and data model agreed

Weeks 3–5: Control API Implementation

- implement dataflow listing, loading, start, stop, and status endpoints
- add basic error handling
- write initial tests
- use both mock and real data to support frontend integration

Milestone

- UI can trigger start and stop actions
- core runtime status is queryable

Weeks 6–8: Run and Monitor View

- integrate WebSocket or SSE
- display runtime status and node summaries
- implement event and log panel
- complete the first end-to-end flow

Milestone

- users can observe dataflow execution live in the browser

Weeks 9–11: Visual Editor

- render graph structure
- implement basic node and edge editing
- support save and export
- connect edited flows back into execution

Milestone

- simple dataflows can be modified and rerun through the GUI

Weeks 12–13: Refinement and Improvements

- fix edge cases
- improve interaction details
- strengthen error feedback
- incorporate mentor feedback

Week 14: Documentation and Finalization

- write usage and architecture documentation
- prepare demos or examples
- finalize testing and delivery

Final Deliverables

- working dora-studio proof of concept
- basic control API
- simple visual editor
- live status view
- documentation

10. Risks and Mitigation

Risk 1: Runtime integration may be more complex than expected

Mitigation: Start with a minimal control API and expose only the most essential operations first.

Risk 2: The visual editor may expand beyond manageable scope

Mitigation: Keep it limited to basic structural editing and reliable configuration round-tripping.

Risk 3: Frontend details may consume too much time

Mitigation: Use a mature framework and component approach, prioritizing functionality over polish.

Risk 4: Runtime status data may not be easy to expose in detail

Mitigation: Start with lifecycle state and basic node summaries, then extend incrementally

where Dora already provides useful signals.

11. Stretch Goals

If the main implementation is completed early and remains stable, I would like to explore one or two of the following:

- more detailed node-level state visualization
- basic validation inside the editor
- structural validation before save or export
- improved demo or example dataflows
- clearer runtime error highlighting

These are optional improvements and will not compromise the main deliverables.

12. Conclusion

Through dora-studio, I want to provide Dora with a practical graphical entry point for understanding, editing, running, and monitoring dataflows. For new users, this would lower the learning barrier. For existing developers, it would improve debugging, experimentation, and demos.

My goal is to build a focused, usable, and extensible first version that can serve as a meaningful foundation for future improvements in the Dora ecosystem.